



IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

Codecamp

Semaine de rentrée – TAF ILSD

1 Règles du *codecamp*

Le but du *codecamp* est de produire en groupe un logiciel qui contient un ensemble de fonctionnalités. Les groupes seront constitués par les enseignants et comporteront entre 4 et 5 élèves.

Chaque groupe construit un logiciel similaire en Python. La première étape commune est décrite ci-après dans la section 2. Pour ce faire, vous pouvez procéder de la façon que vous voulez. Quelques informations sur des éléments Python indispensables figurent en section 3.

Chaque étape doit être validée pour passer à la suivante. La validation inclut une vérification par le groupe du fonctionnement du logiciel, ainsi que la validation du respect des exigences du sujet. Cette relecture du code doit être faite par au moins un membre du groupe. Ensuite, la validation finale doit être attestée par une relecture externe (d'un enseignant ou d'une autre équipe). Il est alors possible de tirer auprès des enseignants l'extension suivante.

Pour éviter de trop perdre de temps, il est recommandé de bien réfléchir à la conception de chaque extension avant de passer à sa réalisation. Nous conseillons également de ne pas imaginer des solutions trop complexes. En effet, le but est de faire le plus d'extensions possible, tout en fournissant un code relativement propre.

De manière générale, il est préférable de ne pas dépendre de bibliothèques externes. On essaie d'utiliser au maximum les mécanismes des bibliothèques standards de Python. En particulier, l'usage de bibliothèques de traitement des données, comme Pandas, est proscrit.

2 Le problème initial

L'objectif de ce *codecamp* est la production d'un logiciel simple permettant de gérer des marque-pages numériques. Le point de départ est commun à tous les groupes et consiste à réaliser les fonctionnalités suivantes : ajout, modification, suppression et affichage de marque-pages. Un marque-page a un *identifiant* fourni par le système lors de son ajout, un *titre*, une *cible* et une zone libre de prise de *notes*.

L'interaction avec l'utilisateur doit rester simple ; nous optons pour une interaction en ligne de commande dans un terminal (CLI – *Command Line Interface*).

Chaque exécution du programme recevra en paramètre un nom de fichier dans lequel sont contenus les marque-pages déjà créés. Le résultat de l'exécution modifiera généralement ce fichier.

Vous êtes libres du choix du format du fichier, mais nous recommandons de rester simple (par exemple, un format texte avec une ligne par marque-page).

Cette première version doit donc permettre d'exécuter les fonctions suivantes :

- `bookmarks marque-pages.txt add <...>` : ajoute au fichier `marque-pages.txt` le nouveau marque-page, puis retourne son identifiant;
- `bookmarks marque-pages.txt modify id <...>` : remplace les informations du marque-page portant l'identifiant `id` dans `marque-pages.txt`, ou renvoie un message si celui-ci n'est pas trouvé;
- `bookmarks marque-pages.txt rm id` : retire la ligne du fichier `marque-pages.txt` contenant le marque-page portant l'identifiant `id`, ou renvoie un message si celui-ci n'est pas trouvé;
- `bookmarks marque-pages.txt show` : affiche la liste des marque-pages enregistrés dans le fichier `marque-pages.txt` sous la forme ci-dessous, en les triant par identifiant.

```
+-----+-----+-----+-----+
| id  | titre | cible | notes |
+-----+-----+-----+-----+
| ... | ...   | ...   | ...   |
+-----+-----+-----+-----+
```

La zone de notes est libre. Vous devez fournir des options que l'utilisateur pourra spécifier afin d'utiliser votre programme (par exemple, `-t "Linux archives" -n "À consulter chaque semaine" "https://lore.kernel.org"`). Les valeurs de certaines informations sont contraintes :

titre maximum 64 caractères

cible une URL valide

3 Quelques outils Python

3.1 La lecture et l'écriture de fichiers

Pour rendre le projet facile à réaliser, nous recommandons d'adopter un format texte simple dans lequel un marque-page est stocké par ligne. On utilisera alors les fonctions Python suivantes :

- `open(filename, mode)`, qui ouvre le fichier dans un mode donné (`r` : lecture, `w` : création du fichier si nécessaire et écriture écrasant le contenu précédent, `a` : création du fichier si nécessaire et écriture de nouvelles données à la fin du fichier).
- `read()` et `readlines()`, qui retournent respectivement le contenu du fichier et une liste de lignes de contenu. ⚠ Le fichier doit être ouvert en mode `r` ; si vous lisez deux fois le contenu du fichier, le comportement ne sera probablement pas celui que vous souhaitez.
- `write(string)`, qui écrit la chaîne de caractère en argument dans le fichier. ⚠ Le fichier doit être ouvert en mode `w` ou `a`.
- `close()`, qui ferme le fichier.

Voici un exemple montrant la manière « pythonique » de traiter des fichiers. Notez que Python appelle tout seul la fonction `close` à la sortie du bloc `with` lorsque vous faites ainsi.

```
# Lecture de fichier
```

```
with open('monfichier.txt', 'r') as f:
    lines = f.readlines()
```

```
# Écriture de fichier
```

```
with open('monfichier.txt', 'w') as f:
    f.write('Mon contenu\n') # \n est le caractère de retour à la ligne
```

3.2 La gestion de ligne de commande

Nous vous suggérons de rester simple et d'utiliser la bibliothèque standard `argparse`. Sa documentation est consultable en ligne, et nous proposons en annexe A une base de travail.

Le principe de cette bibliothèque est le suivant :

- On crée un parseur avec `ArgumentParser(<...>)`;
- On définit un ensemble de paramètres à donner dans la ligne de commande avec la méthode `add_argument(<...>)`;
- On utilise le parseur avec `parse_args()`.

Il y a trois types d'arguments :

1. Les options, qui peuvent être données un peu n'importe quand;
2. Les options dites positionnelles, qui doivent être données à une position particulière;
3. Les sous-commandes, qui elle-mêmes nécessitent un sous-parseur (que l'on peut ajouter avec `add_subparsers(<...>)`).

4 Les extensions

Les différentes extensions sont classées ci-dessous en quatre niveaux. Chaque extension peut provoquer un changement du format de fichier, de l'affichage, du format des commandes, *etc.* Chaque extension est cumulative; une fois tirée, il faut la réaliser en fonction des autres extensions tirées.

La spécification reste un peu floue à dessein; votre première tâche est bien sûr de clarifier le besoin et spécifier l'extension que vous allez réaliser. Penser à *documenter* et à *concevoir*.

- | | |
|-----------------|---|
| Niveau 1 | <ol style="list-style-type: none">a. Détection de doublons : l'ajout d'un marque-page vient au préalable vérifier que son URL n'a pas déjà été ajoutée.b. Ajout d'étiquettes : il est possible d'associer à un marque-page un ensemble d'étiquettes (par exemple, <i>important</i>, <i>travail</i>, <i>etc.</i>).c. Gestion des dates : chaque marque-page a des dates d'ajout et de dernière lecture, automatiquement ajoutées.d. Nombre de consultations : le logiciel peut suivre le nombre de lectures d'un marque-page. |
| Niveau 2 | <ol style="list-style-type: none">a. Gestion d'un journal : toute action exécutée dans le logiciel est consignée dans un fichier de journal (<i>log</i>).b. Gestion de la configuration : le logiciel devient configurable par un fichier de configuration, adapté aux fonctions de votre logiciel. |

- c. Rappel de lecture [*req.* extension 1.d] : le logiciel met en évidence les marque-pages ajoutés depuis longtemps mais jamais consultés.
 - d. Formats d'export : il est possible d'exporter les informations du logiciel dans un ou plusieurs formats de votre choix.
- Niveau 3**
- a. Classification par URL [*req.* extension 1.b] : le logiciel attribue des étiquettes automatiquement en fonction de la cible du marque-pages.
 - b. Statistiques de lecture : le logiciel expose des métriques pertinentes sur les marque-pages (par exemple, nombre de marque-pages et de lectures par nom de domaine ou étiquette, *etc.*).
 - c. Catégories d'étiquettes [*req.* extension 1.b] : il est possible de placer les étiquettes dans des catégories; les catégories elles-mêmes peuvent être catégorisées.
 - d. Recherche et filtrage : il est possible de rechercher et trier un ensemble de marque-pages sur la base de critères (par exemple, mot contenu dans les notes, nom de domaine, étiquettes, *etc.*).
 - e. Contrôle de conformité des fichiers et de leur contenu : le logiciel comporte des mécanismes de contrôle de conformité et d'intégrité des données (les fichiers ont-ils la forme attendue? le contenu des fichiers est-il cohérent? *etc.*).
- Niveau 4**
- a. Temps de lecture : le logiciel estime le temps de lecture des marque-pages, basé sur des heuristiques paramétrables.
 - b. Affichage arborescent [*req.* extension 3.c] : il est possible de définir un ensemble de règles sur les URL, étiquettes et catégories pour afficher les marque-pages de manière arborescente.
 - c. Lectures connexes : le logiciel peut gérer des liens de diverses natures entre marque-pages et suggérer un parcours de lecture en fonction de ceux-ci.
 - d. Synthèse de fichiers : il est possible de combiner plusieurs fichiers de marque-pages entre eux, avec fusion et gestion des conflits.
 - e. Formats d'import [*req.* extension 2.d] : il est possible d'importer les informations depuis un ou plusieurs formats de votre choix; les modifications doivent être répercutées dans le fichier initial.

Les dépendances entre les extensions sont résumées en figure 1. Notez que les extensions peuvent avoir des interactions non triviales les unes avec les autres, indépendamment de ces dépendances.

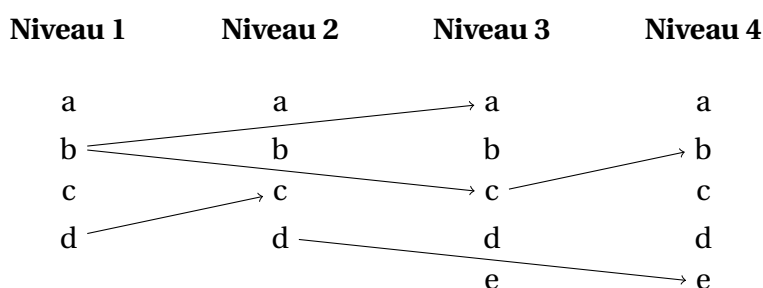


Figure 1 – Dépendances des extensions

5 Critères de validation

1. Qualité du code :

- a. Aucune variable globale ne doit être utilisée, les fonctions prennent toutes leurs données nécessaires en paramètres et retournent tous leurs résultats.
- b. Les fonctions gardent une taille raisonnable.
- c. Il n'y a pas d'élément inutile (fonction ou variable non utilisée par exemple).
- d. Les noms des variables, des fonctions et des fichiers sont pertinents (en adéquation avec leurs contenus et leurs usages). Le code est écrit de manière homogène.
- e. On adopte un style fonctionnel¹ (exercice de style), c'est-à-dire :
 - Les variables utilisées sont non mutables ; une fois créées, elles ne sont pas modifiées.
 - On évite les boucles, on préfère la récursivité et on utilise des itérateurs de type `map()`.
 - Les fonctions n'ont pas d'effet de bord (sauf I/O).
 - Pensez à introduire des contrats/assertions.
 - Pourquoi pas des lambdas pour généraliser certaines fonctions...
- f. Les fonctions sont testées par des tests unitaires fournis.

2. Fonctionnalités :

- a. Le code s'exécute (facilement) sur une machine autre que celle des membres du groupe.
- b. Le code supporte de manière correcte les fonctionnalités annoncées.

3. Documentation :

- a. Chaque fonction du code doit être décrite, ainsi que ses paramètres, résultats, contraintes, effets, son (ou ses) auteur(s)...
- b. Chaque commande utilisable doit être décrite dans une page d'aide (`-h` ou `--help`).
- c. Un *readme* à jour est fourni, il contient une description rapide des fonctionnalités et de leur usage.

4. Process et outils :

- a. Vous utiliserez Git pour travailler en groupe. Un dépôt par groupe.
- b. Vous utiliserez un environnement de développement adapté à la programmation.

1. <https://docs.python.org/3/howto/functional.html>

A Utilisation du parseur

A.1 Exemple de fichier exécutable

Nous supposons que le parseur se trouve dans un fichier `options.py`.

```
#!/usr/bin/env python3
import commands
from options import create_parser

# Création du parseur de ligne de commande
options = create_parser().parse_args()
try:
    # Lecture du fichier de marque-pages, s'il existe
    with open(options.file, 'r') as f:
        bookmarks = f.readlines()
    # Exécution de la commande
    if options.command == 'add':
        commands.add(' '.join(options.details), options.file, bookmarks)
    elif options.command == 'modify':
        commands.modify(options.id, ' '.join(options.details), options.file,
                        bookmarks)
    elif options.command == 'rm':
        commands.rm(options.id, options.file, bookmarks)
    elif options.command == 'show':
        commands.show(bookmarks)

except FileNotFoundError:
    print(f"The file {options.file} was not found")
```

A.2 Exemple de fichier de commandes

Nous supposons que le fichier de commandes est nommé `commands.py` et que les fonctions sont implémentées dans `core.py`.

```
import core

# Implémentation des différentes commandes, utilisant les fonctions contenues dans le module core
def add(details, filename, bookmarks):
    with open(filename, 'a') as f:
        bookmark = core.add(bookmarks, details)
        f.write(bookmark)
    print(f"Successfully added bookmark {bookmark[0]} ({bookmark[1]})")

...
```

A.3 Exemple de parseur

Le code ci-dessous donne une base de travail pour l'implémentation du parseur (`options.py`).

```
import argparse

def create_parser():
    # Création du parseur de ligne de commande
    parser = argparse.ArgumentParser(description='Simple bookmark manager')
    # Ajout d'un argument positionnel (le fichier contenant les marque-pages)
    parser.add_argument('file', help='The bookmarks file')
    # Ajout d'un sous-parseur pour les sous-commandes
    subparsers = parser.add_subparsers(help='The commands to manage bookmarks',
                                       dest='command', required=True)

    # Création du parseur pour la commande add
    parser_add = subparsers.add_parser('add', help='Add a new bookmark.
    The rest of the command line is used for the bookmark details, the default
    being "no details".')
    parser_add.add_argument('details', nargs='*', default="no details",
                           help="bookmark details")

    # Création du parseur pour la commande modify
    parser_modify = subparsers.add_parser('modify', help='Modify a bookmark given
    its id. The rest of the command line is used for the bookmark details, the
    default being "no details"')
    parser_modify.add_argument('id', help="the bookmark id")
    parser_modify.add_argument('details', nargs='*', default="no details",
                              help="the new details")

    # Création du parseur pour la commande rm
    parser_rm = subparsers.add_parser('rm', help='Remove a bookmark given its id')

    parser_rm.add_argument('id', help="the bookmark id")
    # Création du parseur pour la commande show
    parser_show = subparsers.add_parser('show', help='Show the bookmarks')
    return parser
```